



UNIVERSIDAD AUTONOMA DE SINALOA



Licenciatura en Ingeniería de software

Materia:

Administración de sistemas

Practica 10

Nombre del profesor:

Herman Giovany Ayala Zuñiga

Nombre del alumno:

Daniel Humberto Leyva Chavez

Fecha:

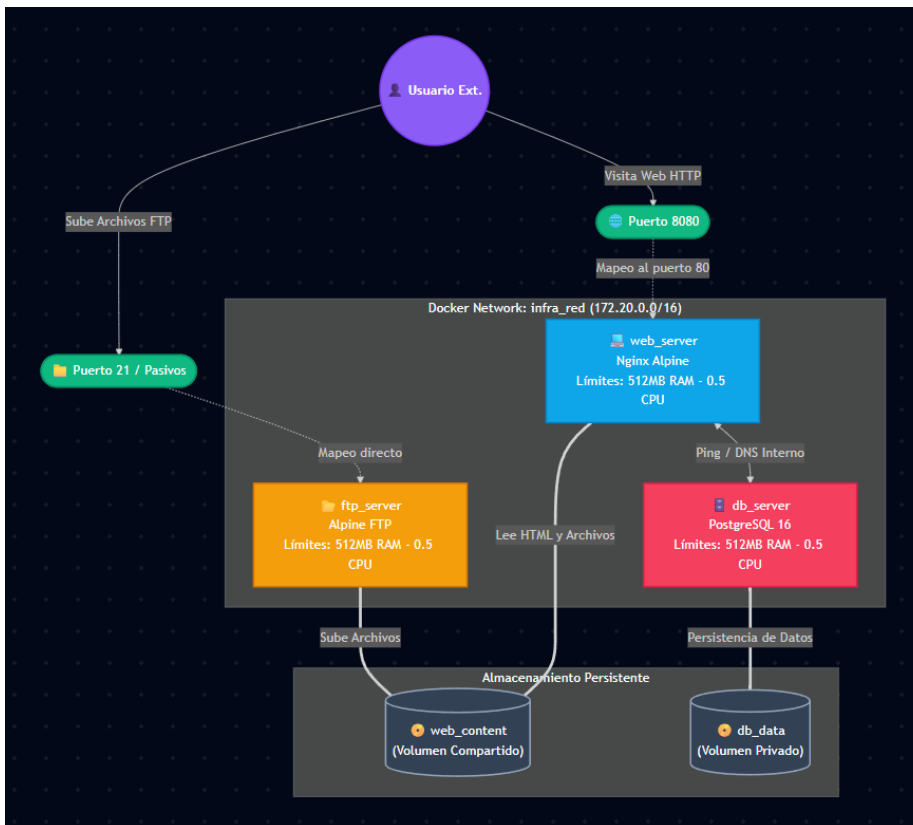
27/04/2026

Commit	Descripción
Commit 1	Creación inicial de scripts modulares (red, volumen, bases de datos).
Commit 2	Corrección de rutas relativas y mejora de seguridad en Dockerfile.
Commit 3	Rediseño de frontend web (estilo Terminal Retro) e integración FTP.

Introducción y Arquitectura

Objetivo: Sustituir las instalaciones locales de servicios por contenedores Docker, automatizando el despliegue de un servidor web Nginx, una base de datos PostgreSQL con respaldos automatizados y un servidor FTP. Todo ejecutándose en una red aislada y con límites estrictos de recursos para garantizar estabilidad y seguridad.

Diagrama de Topología:



Estructura de la infraestructura:

Host Linux: Ejecuta el motor de Docker.

Red: “infra_red” (Subred bridge personalizada `172.20.0.0/16`).

Contenedores:

- “web_server”: Expuesto en el puerto 8080 (mapeado al 80 interno). Basado en Alpine Linux con seguridad reforzada.
- “db_server”: Base de datos aislada internamente (puerto 5432 no expuesto al público).
- “ftp_server”: Expuesto en los puertos 21 y 21000-21010 para modo pasivo de transferencia.

Volúmenes Compartidos: El volumen “web_content” está montado tanto en el FTP como en el servidor Web para reflejar los archivos subidos al instante.

Guia de uso de Scripts:

Requisitos previos:

- Sistema Operativo basado en Linux (Ej. Ubuntu/Debian).
- Conexión a internet para la descarga de dependencias e imágenes de Docker.
- Ejecutar el script principal desde un usuario que cuente con privilegios de administrador (“sudo”).

Instrucciones de ejecución:

Para instalar, configurar y levantar toda la infraestructura con un solo comando, ejecuta: “bash practica10.sh -i”

Flujo de interacción:

El script es totalmente silencioso y automatizado para la instalación. Sin embargo, para destruir el entorno (opción '-r'), el sistema exigirá una confirmación manual para evitar pérdida de datos accidental:

[INFO] Esto eliminara contenedores, red y volúmenes

¿Estás seguro? (s/N):

Se debe escribir `s` y presionar Enter para confirmar.

Parámetros disponibles del Script:

- '-i': Instalar y levantar todos los servicios.
- '-v': Verificar estado de contenedores y consumos de recursos.
- '-s': Detener (stop) todos los contenedores sin borrar sus datos.
- '-u': Iniciar (up) contenedores previamente detenidos.
- '-b': Forzar un respaldo manual de la base de datos PostgreSQL.
- '-r': Reiniciar entorno completo (Peligro: Elimina contenedores, red y volúmenes).

Bitácora de desarrollo y configuración:

Explicación lógica del script:

Se optó por una arquitectura de scripting modular, donde 'practica10.sh' funciona como un orquestador que manda a llamar módulos ubicados en 'lib/Dockers/':

- '**docker_install.sh**': Se encarga de verificar la existencia del motor Docker. Si no existe, lo instala, arranca el servicio y configura las reglas del cortafuegos ('ufw'). También instala 'cronie'.

- '**docker_network.sh**' y '**docker_volumenes.sh**': Ejecutan las sentencias para aislar la red ('172.20.0.0/16') y preparan los volúmenes físicos ('db_data', 'web_content').

- '**docker_web.sh**': Manda a llamar al comando 'docker build' en base al 'Dockerfile' personalizado. Este Dockerfile elimina firmas ('server_tokens off;') y ejecuta los procesos workers de Nginx con el usuario sin privilegios 'webuser'. Luego lanza el contenedor con la flag '--memory 512m' y '--cpus 0.5'.

- '**docker_db.sh**': Levanta el motor PostgreSQL e inyecta usuarios y contraseñas provenientes del archivo 'claves.txt'. A continuación, utiliza un Here-Doc para inyectar un script autogenerado que programa un 'cron' diario en el host (a las 2:00

AM) realizando respaldos por medio de 'pg_dump' hacia la ruta '/opt/docker_backups'.

- '**Docker_ftp.sh**': Lanza el contenedor de FTP compartiendo el mismo volumen 'web_content' utilizado por Nginx.

Protocolos de pruebas:

10.1 Persistencia BD:

- Borrar contenedor BD con `docker rm -f db\_server` y reiniciar
- Los datos en PostgreSQL siguen existiendo al reconectar

```
dleyva@Srv-Linux:~> docker exec -it db_server psql -U admin -d tarea10 -c "CREATE TABLE test (dato TEXT);"
ERROR:  relation "test" already exists
dleyva@Srv-Linux:~> docker exec -it db_server psql -U admin -d tarea10 -c "INSERT INTO test VALUES ('Diego');"
INSERT 0 1
dleyva@Srv-Linux:~> docker exec -it db_server psql -U admin -d tarea10 -c "SELECT * FROM test;"
      dato
-----
 Daniel
 Diego
(2 rows)

dleyva@Srv-Linux:~> docker rm -f db_server
db_server
dleyva@Srv-Linux:~> docker exec -it db_server psql -U admin -d tarea10 -c "SELECT * FROM test;"
Error response from daemon: No such container: db_server
```

Se creo una tabla y se insertaron datos, luego se eliminó el Docker y se intentó verificar y marco error.

```
==== Uso de recursos ====

CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O     PIDS
548b9fa08d20   db_server     0.04%    17.09MiB / 512MiB    3.34%    426B / 126B    0B / 1.03MB    6
77b80284677c   ftp_server    0.02%    1.16MiB / 512MiB     0.23%    1.27kB / 126B  0B / 1.53MB    3
51d7b9be5437   web_server    0.00%    6.824MiB / 512MiB    1.33%    2.07kB / 126B  8.28MB / 4.1kB  4
dleyva@Srv-Linux:~/administracion_sistemas/Scripts/linux/tarea10> docker exec -it db_server psql -U admin -d tarea10 -c
"SELECT * FROM test;"
      dato
-----
 Daniel
 Diego
(2 rows)
```

Se recreo el Docker y al hacer la prueba de nuevo hubo persistencia.

10.2 Aislamiento Red

- 'docker exec web_server ping db_server'
- Respuestas de ping exitosas usando el nombre del contenedor

```
dleyva@Srv-Linux:~/administracion_sistemas/Scripts/linux/tarea10> docker exec web_server ping db_server
PING db_server (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.325 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.071 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.081 ms
64 bytes from 172.20.0.3: seq=3 ttl=64 time=0.097 ms
64 bytes from 172.20.0.3: seq=4 ttl=64 time=0.094 ms
^Cdleyva@Srv-Linux:~/administracion_sistemas/Scripts/linux/tarea10> docker exec web_server ping web_server
PING web_server (172.20.0.4): 56 data bytes
64 bytes from 172.20.0.4: seq=0 ttl=64 time=0.116 ms
64 bytes from 172.20.0.4: seq=1 ttl=64 time=0.142 ms
64 bytes from 172.20.0.4: seq=2 ttl=64 time=0.173 ms
64 bytes from 172.20.0.4: seq=3 ttl=64 time=0.155 ms
^Cdleyva@Srv-Linux:~/administracion_sistemas/Scripts/linux/tarea10> docker exec web_server ping ftp_server
PING ftp_server (172.20.0.2): 56 data bytes
64 bytes from 172.20.0.2: seq=0 ttl=64 time=0.528 ms
64 bytes from 172.20.0.2: seq=1 ttl=64 time=0.204 ms
64 bytes from 172.20.0.2: seq=2 ttl=64 time=0.090 ms
64 bytes from 172.20.0.2: seq=3 ttl=64 time=0.069 ms
64 bytes from 172.20.0.2: seq=4 ttl=64 time=0.301 ms
```

10.3 Permisos FTP

- Subir archivo vía cliente FTP y abrir la IP en el puerto 8080
- El archivo se lista o visualiza directamente en el navegador

..				
css				
img				
50x.html	4...	Brave HTML Doc...	07/04/2026 05:09:00 a. m.	-rw-r--r-- 0 0
index.html	4...	Brave HTML Doc...	27/04/2026 03:42:00 p. m.	-rw-r--r-- 1000 1000
Tabla_de_tipos_POKEMON.jpg	7...	Archivo JPG	27/04/2026 03:44:00 p. m.	-rw-r--r-- 1000 1000

FTP vacío.

..				
css				
img				
50x.html	4...	Brave HTML Doc...	07/04/2026 05:09:00 a. m.	-rw-r--r-- 0 0
AlcanceDelProyecto.pdf	1...	Brave HTML Doc...	27/04/2026 05:10:00 p. m.	-rw-r--r-- 1000 1000
CasosDeUsoEquipo1.pdf	1...	Brave HTML Doc...	27/04/2026 05:10:00 p. m.	-rw-r--r-- 1000 1000
CasosDeUsoEquipo2.pdf	1...	Brave HTML Doc...	27/04/2026 05:10:00 p. m.	-rw-r--r-- 1000 1000
index.html	4...	Brave HTML Doc...	27/04/2026 03:42:00 p. m.	-rw-r--r-- 1000 1000
Tabla_de_tipos_POKEMON.jpg	7...	Archivo JPG	27/04/2026 03:44:00 p. m.	-rw-r--r-- 1000 1000

Se agregaron 3 PDF.

```
root@linux-server:~# ./init_practica10.sh

Iniciando sistema para el administrador: Leyva Chavez Daniel Humberto

--- ESTADO DE CONTENEDORES DOCKER ---

[ SERVIDOR WEB ] [ BASE DE DATOS ] [ SERVIDOR FTP ]
IMG: Nginx Alpine  IMG: PostgreSQL 16  IMG: Alpine FTP
PORT: 8080->80      VOL: db_data      VOL: web_content
STAT: Online        STAT: Online      STAT: Online

--- SISTEMA DE ARCHIVOS COMPARTIDO FTP/WEB ---

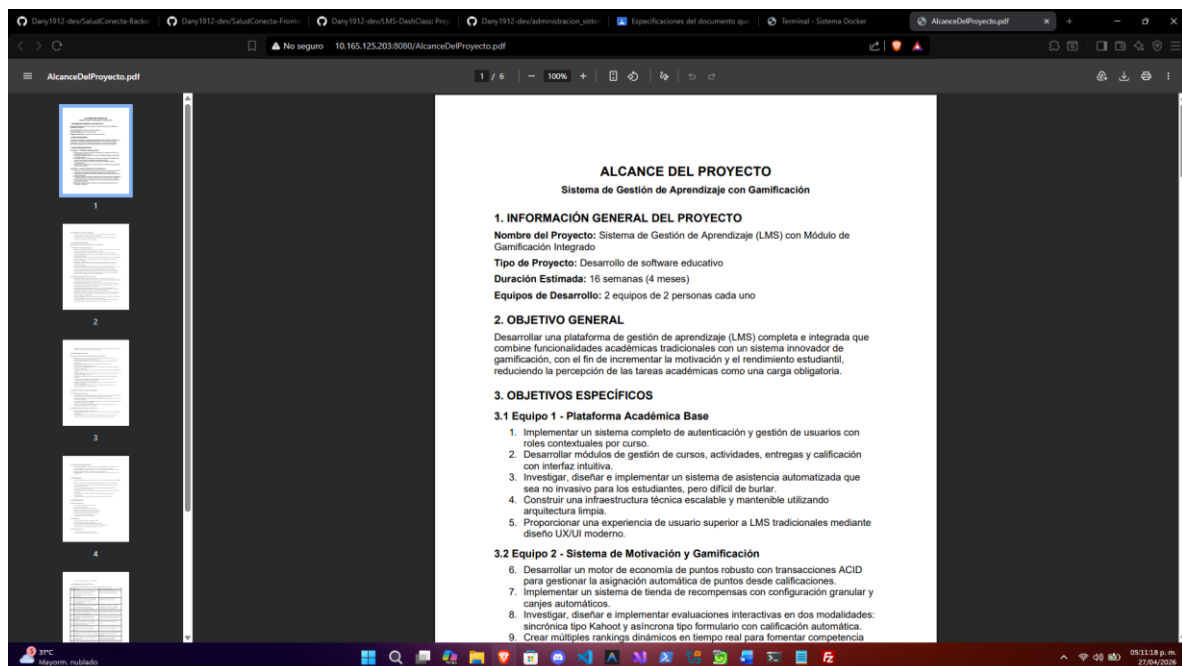
Volumen enlazado detectado. Todo archivo cargado vía FTP al servidor se desplegará aquí.
Ingresa el nombre del archivo para visualizarlo en el navegador:

root@web-client:~# GET / AlcanceDelProyecto [EJECUTAR]

root@linux-server:~# status_check

Todo el sistema operando al 100%. Presione CTRL+C para salir
```

Página web.



Prueba completa.

10.4 Límites Recursos

- Ejecutar 'bash practica10.sh -v'
- Tabla de 'docker stats' mostrando un 'LIMIT' de memoria de 512MiB

```
^Cdleyva@Srv-Linux:~/administracion_sistemas/Scripts/Linux/tarea10$ bash practica10.sh -v

==== Estado de contenedores ====

NAMES          STATUS          PORTS
db_server      Up 10 minutes   5432/tcp
ftp_server     Up About an hour 0.0.0.0:21->21/tcp, [::]:21->21/tcp, 0.0.0.0:21000-21010->21000-21010/tcp, [::]:21000-21010->21000-21010/tcp
web_server     Up About an hour 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

==== Uso de recursos ====

CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O     PIDS
548b9fa08d20   db_server     0.87%    18.13MiB / 512MiB   3.54%    28.7kB / 27.7kB   0B / 1.26MB   6
77b80284677c   ftp_server    0.05%    2.191MiB / 512MiB   0.43%    504kB / 33.2kB    336kB / 2.01MB 5
51d7b9be5437   web_server    0.08%    7.453MiB / 512MiB   1.46%    62.2kB / 200kB    8.28MB / 4.1kB  7
```

Lecciones Aprendidas:

Durante el desarrollo de esta práctica, uno de los retos técnicos más grandes consistió en el manejo correcto del **Contexto de Construcción (Build Context)** de Docker y el enrutamiento relativo en Bash. Al separar la lógica de instalación ('lib/Dockers/') de los archivos front-end y Dockerfile ('tarea10/'), se tuvo que calibrar minuciosamente cómo se copiaban los archivos web para no exponer datos confidenciales (como 'claves.txt') en el servidor público.

Además, se consolidó el conocimiento sobre seguridad de contenedores; en lugar de descargar una imagen genérica, se comprobó la eficacia de interceptar la configuración de Nginx mediante 'sed' directamente en el 'Dockerfile' para inhabilitar firmas y asignar usuarios no administrativos ('webuser').

Github: https://github.com/Dany1912-dev/administracion_sistemas/commits/main/